

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №3
по дисциплине «Вычислительная математика»
Тема: численное интегрирование

Студент гр. 4341

Четвертная В.Л.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Целью лабораторной работы является изучение методов прямоугольников, трапеций и Симпсона для приближённого вычисления определённых интегралов и написание программы, реализующей данные методы.

Задание.

Требуется вычислить определённый интеграл с заданной точностью Еpsilon тремя методами, применяя формулу Рунге для практической оценки погрешности. Провести исследование скорости сходимости методов.

Вариант 23: $\int_1^2 e^x/x$

Теоретический минимум

Постановка задачи численного интегрирования. Задача численного интегрирования состоит в приближённом вычислении определённого интеграла $I = \int_a^b f(x) dx$ в случаях, когда первообразная не выражается через элементарные функции или её нахождение затруднительно. Для этого отрезок интегрирования $[a,b]$ разбивают на n равных частей с шагом $h = (b - a)/n$, образуя узлы сетки $x_i = a + ih$ ($i=0,1,\dots,n$). На полученных частичных отрезках подынтегральную функцию заменяют более простой (интерполяционным многочленом) и интегрируют, что приводит к квадратурным формулам.

Составная формула прямоугольников. Составная формула прямоугольников (средних) использует значение функции в середине каждого частичного отрезка: $I_{\text{rect}} = h \sum_{i=0}^{n-1} f(x_i + h/2)$. Эта формула имеет порядок точности $O(h^2)$.

Составная формула трапеций. Составная формула трапеций заменяет функцию на каждом отрезке линейной функцией

(хордой): $I_{\text{trap}} = h \left(\frac{f(a)+f(b)}{2} + \sum_{i=1}^{n-1} f(x_i) \right)$. Её порядок точности также составляет $O(h^2)$.

Составная формула Симпсона. Формула Симпсона (парабол) требует чётного числа отрезков n и на каждом сдвоенном отрезке заменяет функцию квадратичной параболой, проходящей через три точки:

$$I_{\text{simp}} = \frac{h}{3} \left[f(a) + f(b) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}) + 2 \sum_{i=1}^{n/2-1} f(x_{2i}) \right]$$
. Благодаря использованию парабол эта формула имеет более высокий порядок точности $O(h^4)$.

Правило Рунге для оценки погрешности. Для практической оценки погрешности применяют правило Рунге: проводят вычисления на сетках с шагом h и $2h$, получая приближённые значения I_h и I_{2h} . Уточнённое значение

интеграла вычисляется как $I_{\text{уточн}} = I_h + \frac{I_h - I_{2h}}{2^p - 1}$, а погрешность принимается

равной $\Delta = \frac{|I_h - I_{2h}|}{2^p - 1}$, где $p=2$ для формул прямоугольников и трапеций, $p=4$ для формулы Симпсона.

Достижение заданной точности. Последовательно удваивая число разбиений n (уменьшая шаг h) и оценивая погрешность по правилу Рунге, можно добиться выполнения условия $\Delta < \epsilon$ для заданной точности ϵ . При этом для каждой квадратурной формулы определяется наименьшее n (наибольший шаг h), обеспечивающее требуемую точность.

Выполнения программы.

Программа написана на языке C++.

Содержит следующие ключевые функции:

- `double f(double x)` – вычисление подынтегрального выражения $\exp(x)/x$.
- `double rectangle(...)`, `double trapeze(...)`, `double simpson(...)` – реализация составных квадратурных формул с постоянным шагом.
- `double runge(double I_h, double I_2h, int m_num)` – оценка погрешности по правилу Рунге. Учтены порядки точности: $1/3$ для методов

прямоугольников и трапеций (порядок $p=2$), $1/15$ для метода Симпсона (порядок $p=4$).

- `double integrate(...)` – управляющая функция. Начинает с $n=2$, удваивает сетку пока ошибка Рунге не станет меньше ϵ .

Код программы приведён в Приложении А.

Тестирование.

3

Rectangle:

steps: 16

result: 3.059

error: 0.0003

Trapeze:

steps: 16

result: 3.060

error: 0.0006

Simpson:

steps: 4

result: 3.059

error: 0.0001

4

Rectangle:

steps: 32

result: 3.0590

error: 0.00008

Trapeze:

steps: 64

result: 3.0592

error: 0.00004

Simpson:

steps: 4

result: 3.0592

error: 0.00009

6

Rectangle:

steps: 512

result: 3.059116

error: 0.0000003

Trapeze:

steps: 512

result: 3.059117

error: 0.0000006

Simpson:

steps: 16

result: 3.059117

error: 0.0000005

Выводы.

В ходе лабораторной работы были изучены и реализованы методы прямоугольников, трапеций и Симпсона с автоматическим дроблением шага по правилу Рунге. Все три метода успешно вычислили интеграл с заданной точностью. Исследование показало, что метод Симпсона обеспечивает значительно более высокую точность при равных вычислительных затратах (числе разбиений) по сравнению с методами прямоугольников и трапеций, что

соответствует теоретическим данным. Практическая оценка погрешности по Рунге подтвердила свою эффективность, позволяя динамически останавливать счёт при достижении требуемой точности.

ПРИЛОЖЕНИЕ А.

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
#include <iostream>
#include <iomanip>
#include <cmath>

double f(double x){
    return std::exp(x)/x;
}

double rectangle(double a, double b, int n){
    double h=(b-a)/n;
    double sum=0.0;

    //  $\sum f(x_{i+h/2}), i = 0..n-1$ 
    for (int i=0; i < n; i++){
        double x=a+(i+0.5)*h;
        sum += f(x);
    }

    return sum*h;
}

double trapeze(double a, double b, int n){
    double h=(b-a)/n;
    double sum=0.5*(f(a)+f(b));

    //  $(f(a)+f(b))/2 + \sum f(a+i*h), i = 1..n-1$ 
    for (int i=1; i < n; i++)
        sum += f(a+i*h);

    return sum*h;
}

double simpson(double a, double b, int n){
    double h=(b-a)/n;
    double sum=f(a)+f(b);

    //  $4*\sum f(x_{\{2i-1\}}), i = 1..n/2$ 
    for (int i=1; i < n; i += 2)
        sum += 4.0*f(a+i*h);

    //  $2*\sum f(x_{\{2i\}}), i = 1..n/2-1$ 
    for (int i=2; i < n-1; i += 2)
        sum += 2.0*f(a+i*h);

    return sum*h/3.0;
}

double runge(double I_h, double I_2h, int m_num){
    if (m_num == 3)
        return std::fabs(I_h-I_2h)/15.0;

    return std::fabs(I_h-I_2h)/3.0;
}
```

```

    double integrate(double a, double b, double e, int m_num, double&
err, int& n){
        double I_h=(m_num == 1 ? rectangle(a, b, n) : (m_num == 2 ?
trapeze(a, b, n) : simpson(a, b, n)));
        double I_2h=(m_num == 1 ? rectangle(a, b, 2*n) : (m_num == 2 ?
trapeze(a, b, 2*n) : simpson(a, b, 2*n)));

        err = runge(I_h, I_2h, m_num);
        while (err > e){
            n *= 2;

            I_h = I_2h;
            I_2h = (m_num == 1 ? rectangle(a, b, 2*n) : (m_num == 2 ?
trapeze(a, b, 2*n) : simpson(a, b, 2*n)));

            err = runge(I_h, I_2h, m_num);
        }

        n *= 2;
        return I_2h;
    }

int main(){
    double a=1.0, b=2.0;

    int t;
    std::cin >> t;

    double e=std::pow(10, -t);

    int n=2;
    double err, res;

    res = integrate(a, b, e, 1, err, n);
    std::cout << "Rectangle:\nsteps: " << n << "\nresult: " <<
std::fixed << std::setprecision(t) << res << "\nerror: " <<
std::setprecision(t+1) << err << "\n\n";

    n = 2;
    res = integrate(a, b, e, 2, err, n);
    std::cout << "Trapeze:\nsteps: " << n << "\nresult: " <<
std::fixed << std::setprecision(t) << res << "\nerror: " <<
std::setprecision(t+1) << err << "\n\n";

    n = 2;
    res = integrate(a, b, e, 3, err, n);
    std::cout << "Simpson:\nsteps: " << n << "\nresult: " <<
std::fixed << std::setprecision(t) << res << "\nerror: " <<
std::setprecision(t+1) << err << "\n\n";
}

```